

[L4] Chapters 7 & 8

Friday, April 24, 2020 5:00 PM

Real-Time Computing

- Came to Europe not through the military or airline reservations but through banks
 - Banks like Barclays established cash dispensers (1967)
- Nobody really needed a real-time response from a computer at that time. Jobs could be done in batch.
- Most people also didn't trust it
- Once Whirlwind was completed, the involved companies needed to sell such real-time machines to capitalise on their innovations
- In Europe, banks were the first who wanted this
- With the appearance of the card, real-time became necessary. Banks and customers needed to know at any point whether there was money in an account available for a transaction
- Barclays was presenting itself as a modern bank
 - 1961 they already had batch jobs running at their computer center
 - They took the next step by installing the cashier and now needed real-time computing
 - This was a step towards the cashless society
- Barclays had several offices throughout the country and needed a central computer node that would store, manage and distribute information about bank accounts
- Burroughs was involved in SAGE (the D825 SAGE computer) and knew how to make real-time computers
 - They created the B5500 (huge mainframe) and the TC500 (could be used to link to the B5500 for I/O)
 - 1965 they made a huge sale to US Steel Detroit and one to NORAD in Colorado Springs
 - They presented themselves as the company of the future "Bringing business automation to far-sighted companies", trying to compete with IBM in this sector
- Barclays and Burroughs entered a partnership.
 - Burroughs already had a factory in the UK that it had acquired through a merger with some smaller European manufacturers
- 1965 Burroughs announced the arrival of the B8500 / TC500 combination. It was due in 1969.
- In order to convince Barclays to buy their machines, Burroughs invited the bankers to the secret Colorado Springs military base where they had one of the B7500 computers running
 - This impressed the bankers and convinced them that Burroughs had everything under control
- The machines were delayed and Burroughs agreed to install some B6500's instead and would come ahead of time and start testing with some B5500's to see whether the communications were working
- 1968 Barclays merged with Martins and now had an even higher need for the machines
- 1969 the production of the B8500 was cancelled
- The B5500's and B6500's in the computer center had a problem too: The vibrations from the nearby trains caused problems with the machines' hardware
- Barclays became nervous and started trying to "limp in batch mode"
- They had to switch from being real-time to being on-time
- Despite the sluggish progress that Barclays and Burroughs were making, the banking sector was still the one pushing for real-time computing in Europe

The UPC

In the US:

- 1973 it was adopted and it was a political feat
- It was a necessity for larger companies. For smaller companies, it was an extra burden when trying to enter the market
- It proved the growing trust in these machines

In Europe:

- Decided not to join the Universal Product Code
- They created the Article Number Code (1973-1974), in the meanwhile known as the International Product Code (IPC)
- It had 2 more digits than the American version

- It was a political choice and brought the European countries closer together, creating a common market
- It was also a stand against the dominance of American technology

Agenda's

What people want to achieve

Examples:

- Selling machines
- Achieving efficiency in the office
- Even within the academic world, there were different agendas
- The academic world was very localised. Researchers in different areas of the world were focusing on completely different things. While the dutch scientists were preoccupied with waterworks and airplanes, scientists in the US were dealing with atomic weapons. Sometimes there were different agenda's even within the same country or university

There were 5 main agenda's within the academic world:

- Cybernetics (Norbert Wiener, Claude Shannon)
 - Interested in AI, but not only
- Logic (Turing, Church)
- Sharing information (people from CERN or people in the field of astronomy)
- Ordering, sorting, processing (administration business, economics, business analytics, business-oriented research in general)
- Calculations

The first machines weren't being invented for the sake of inventing computers in the modern sense of the world. They were a means to a purpose, they were meant to solve a problem. The fact that we call them computers is a result of hindsight

Even in 1967, when some real-time installations were already up and running, some (ex. Dijkstra) still doubted the usefulness of such applications. He wanted a field of computer science determined by mathematical machines

Programs and Software

- Wiring on a plugboard or punched tape represent programs like we write them today, but they are very far removed from it
- It would be very hard to understand the program by looking at the tape / plugboard
- Feedback was received through audio

In other words, programming was very different and very far removed from what we do to day

- Programs were things that you **prepared**, that you **assembled**
- The electrologica X8 had a simpler way of programming, through a 3-row switch array
- At the time there were no alternatives to these approaches

With project whirlwind, people started thinking of more efficient alternatives: autocoding

- FORTRAN (1957)
- COBOL (1959)

Europe had an alternative: ALGOL 60

- In the business application world, it was never successful
- As an academic language, it was THE language of the 1960s (in Europe)
 - Elegant
 - Universal (ideal: both for machines and the language used for programming)
 - It satisfied a sense of clarity and order
- From the US point of view it was academic, not flexible (better in theory than in practice) and difficult to learn
- What was the problem that it was trying to solve?
 - Programs would work on the machine they were made for
 - When a new machine would come out it had to be rewritten or adapted in some clever way
 - **TO-DO**: ease of programming, make existing programs run on a new machine
 - Lots of people were interested all over the world:
 - ACM
 - IEEE
 - IBM

- IFIP
 - GAMM
 - SHARE (IBM)
 - USE (Univac)
 - DUO (Datatron)
 - CUBE (Burroughs)
- Why 3 languages?
 - For some, any language would do (ACM, IEEE). ACM adopted ALGOL rapidly, IEEE was quite happy with FORTRAN and could care less.
 - IBM looked at what would sell best. FORTRAN and COBOL had the best market shares. ALGOL didn't bring anything new. It was good at calculations, but COBOL could do the job too
 - IFIP went with the flow. They offered a platform to ALGOL
 - User groups were not enthusiastic about ALGOL. Therefore, the groups' companies went along with the group. They had no reason to promote a different language
 - GAMM believed there should be a universal language. This was supposed to be ALGOL
- A lot of effort went into making ALGOL 60
- Dijkstra wrote a compiler for the X1 and X8
- 1962 IFIP created working group 2.1 to work on ALGOL
- The ACM decided that ALGOL would be the standard for publication of scientific algorithms (but not other algorithms). This let ALGOL into the american market but also kept it from getting really big. This also did not last very long

Rise of the software industry

- At the start of the 60s, there was no money in software
- At the time, software was called "code"
- Software companies wrote code, but it was not enough to support a business. They also did computer maintenance, training, etc
- A program was not software, it was code. A company testing code was considered a software project. => Software had a different meaning
- With the unbundling decision of IBM, the term changed:
 - Soft: not the machine itself. It was something you could read, could write, could think and reason about
 - Ware: salable, an economical good
- Software no longer came with the machine, it was no longer free.
- People would do it in their spare time, on the side. Dijkstra wrote ALGOL compilers on the side, it was an extra, not the essence of computing.
- The european software industry relied a lot on the work done by academics in their spare time, who refused to be paid for something they did for fun on the side

The software crisis

- Programs were growing in length and the number of people working on it was always increasing
- Code was rarely fool-proof, a crisis was proclaimed. According to Campbell-Kelly, this was done because it helped american academics towards their goal of building out IT professionalism. There was no crisis, according to Kelly
- In europe, a similar "crisis" occurred
 - When there was a problem, there was usually at least a company offering a solution
 - In the academic world, there were problems with getting computer science incorporated within universities. These new fields were part of either an engineering program, a mathematical program or a program within the social sciences. Academics were eager to get rid of that financing balast, which is easier in a "crisis"
 - While in the US, university programs can be created at will, given that there are people who want it and funds to run it, in Europe they were government funded, so the government had to approve something new. Therefore, the need for these new computer scientists had to be explained, which was not evident in the 60s. Computer scientists were there, but they were engineers, mathematicians or social scientists. The field of computer science did not exist.
- Software engineering appeared (the "engineering" in the name proves that there was already an agenda) as a way of solving a crisis:
 - It implied that software was approachable in the same way as a large engineering project. This made sense to engineers and mathematicians; not so much to social scientists, but they were a 2:1 minority.
 - It made sense and was the way out. It could convince governments to pay
 - 1968 in Garmisch-Partenkirchen, several people came together from various fields, with the intent of proclaiming a

crisis (GAMM, Universities, IFIP, ACM)

- The only group with experience with the crisis at hand was not present: IBM
- If they were going to proclaim a crisis, they would need more than a buzzword: they would need to decide what software engineering should entail. Several options were proposed:
 - ALGOL as a universal programming language (supported especially by GAMM). This was dismissed.
 - Structured programming was not dismissed. It was preferred by a lot of europeans, especially mathematicians.
 - Project management was also not dismissed.
- Computer science became a science. Software engineering meant structured programming (preferred in Europe) and project management (preferred in America)
- Programming had to become "the end of a vocation" it had to become professionalised